

GlamCart

Beauty & Cosmetics E-Commerce Platform

Next.js • Flutter • Node.js / Express

SOFTWARE REQUIREMENTS SPECIFICATION

SRS Document

VERSION

1.0

DATE

2026-05-08

AUTHOR

Hamja Paldiwalla

STATUS

Active

Table of Contents

1. Introduction

- 1.1 Purpose
- 1.2 Scope
- 1.3 Intended Audience
- 1.4 Demo Credentials

2. System Overview

- 2.1 Architecture
- 2.2 Base URLs

3. Technology Stack

- Backend
- Web Frontend
- Mobile (Flutter)

4. Database Schema

- 4.1–4.11 All Models

5. Backend API Specification

- 5.1 Middleware
- 5.2 Auth
- 5.3 Products
- 5.4 Categories
- 5.5 Brands
- 5.6 Cart
- 5.7 Wishlist
- 5.8 Orders
- 5.9 Users
- 5.10 Coupons
- 5.11 Payments
- 5.12 Health

6. Web Frontend Specification

- 6.1–6.9 Pages & Components

7. Mobile App Specification

- 7.1–7.5 Structure, Models, API, Screens, Providers

8. State Management

- 8.1 Web (React Context)
- 8.2 Mobile (Provider Pattern)

9. Validation Rules

10. Security

- 10.1–10.5 Auth, Passwords, Data, API, Input

11. Key User Flows

11.1–11.5 Registration to Session Restore

12. Error Handling

12.1–12.4 Web, Flutter, UI States, Logger

13. Known Limitations

1 Introduction

1.1 Purpose

This document describes the complete software requirements for **GlamCart**, a multi-platform beauty e-commerce application inspired by Nykaa. It covers the backend REST API, web frontend (Next.js), and mobile app (Flutter).

1.2 Scope

GlamCart allows users to:

- Browse and search beauty products across categories
- Add products to cart and wishlist
- Apply discount coupons
- Place orders with multiple payment methods (COD and Razorpay)
- Manage delivery addresses and order history

1.3 Intended Audience

- Developers maintaining or extending the codebase
- Reviewers evaluating the project
- Testers writing test cases

1.4 Demo Credentials

Field	Value
Email	demo@glamcart.com
Password	Demo@1234
Coupon	GLAMCART10

2 System Overview

2.1 Architecture

Next.js Web App (port 3000) + Flutter Mobile App (iOS / Android) | | +----- HTTP/JSON -----+
| Express REST API (port 5002) JWT Auth + Razorpay | Prisma ORM | PostgreSQL Database (nykaa_latest)

2.2 Base URLs

Service	URL
Backend API	http://localhost:5002/api
Web Frontend	http://localhost:3000

3 Technology Stack

3.1 Backend

Component	Technology
Runtime	Node.js 18+
Framework	Express.js 4.x
Database	PostgreSQL 14+
ORM	Prisma 5.x
Authentication	JWT (jsonwebtoken) + bcryptjs
Payment	Razorpay API
Dev Server	nodemon

3.2 Web Frontend

Component	Technology
Framework	Next.js 14 (App Router)
UI	React 18 + Tailwind CSS 3
HTTP Client	Axios 1.x (10s timeout)
State	React Context API
Notifications	react-hot-toast
Icons	react-icons

3.3 Mobile (Flutter)

Component	Technology
Framework	Flutter 3.10+ (Dart)
State Management	Provider 6.x (ChangeNotifier)
HTTP Client	Dio 5.x (10s timeout)
Local Storage	SharedPreferences 2.x
Image Caching	cached_network_image 3.x
Loading UI	shimmer 3.x
Badges	badges 3.x
Payment	razorpay_flutter 1.x
Date Formatting	intl 0.20.x

4 Database Schema

4.1 User

Field	Type	Constraints
id	Int	PK, auto-increment
email	String	Unique, required
password	String	Required, bcrypt hashed
name	String	Required, min 2 chars
phone	String?	Optional, 10 digits
avatar	String?	Optional, URL
role	String	Default: USER (USER ADMIN)
createdAt	DateTime	Auto-set
updatedAt	DateTime	Auto-update

Relations: addresses[], orders[], cart[], wishlist[], reviews[]

4.2 Address

Field	Type	Constraints
id	Int	PK, auto-increment
userId	Int	FK → User
type	String	Default: HOME (HOME WORK OTHER)
name	String	Required, min 2 chars
phone	String	Required, 10 digits
line1	String	Required
line2	String?	Optional
city	String	Required
state	String	Required
pincode	String	Required, 6 digits
isDefault	Boolean	Default: false — only one default per user

4.3 Category

Field	Type	Constraints
id	Int	PK, auto-increment
name	String	Unique

slug	String	Unique, kebab-case
description	String?	Optional
image	String?	Optional, URL
parentId	Int?	FK → Category (self-reference)
createdAt	DateTime	Auto-set

Self-referencing tree supports parent → children category hierarchy.

4.4 Brand

Field	Type	Constraints
id	Int	PK, auto-increment
name	String	Unique
slug	String	Unique, kebab-case
description	String?	Optional
logo	String?	Optional, URL
website	String?	Optional, URL
createdAt	DateTime	Auto-set

4.5 Product

Field	Type	Constraints
id	Int	PK, auto-increment
name	String	Required
slug	String	Unique, kebab-case
description	String?	Optional, long text
price	Float	Selling price, required
mrp	Float	Original/marked price
discount	Float	Default: 0
images	JSON	Array of URLs, default: []
stock	Int	Default: 0
sku	String?	Unique if provided
rating	Float	Default: 0, range: 0-5
reviewCount	Int	Default: 0
categoryId	Int	FK → Category
brandId	Int	FK → Brand

tags	String?	Comma-separated search tags
isActive	Boolean	Default: true
isFeatured	Boolean	Default: false
isBestSeller	Boolean	Default: false
createdAt	DateTime	Auto-set
updatedAt	DateTime	Auto-update

4.6 CartItem

Field	Type	Constraints
id	Int	PK, auto-increment
userId	Int	FK → User
productId	Int	FK → Product
quantity	Int	Default: 1, min: 1
shade	String?	Optional variant
size	String?	Optional variant

Composite unique (userId, productId) — quantity increments on re-add.

4.7 WishlistItem

Field	Type	Constraints
id	Int	PK, auto-increment
userId	Int	FK → User
productId	Int	FK → Product
addedAt	DateTime	Auto-set

Composite unique (userId, productId).

4.8 Order

Field	Type	Constraints
id	Int	PK, auto-increment
userId	Int	FK → User
addressId	Int?	FK → Address, optional
status	String	PENDING CONFIRMED PROCESSING SHIPPED DELIVERED CANCELLED
paymentStatus	String	PENDING PAID FAILED REFUNDED
paymentMethod	String	COD UPI CARD NETBANKING
subtotal	Float	Sum of item prices

discount	Float	Default: 0, coupon discount
deliveryCharge	Float	Rs.0 if subtotal >= Rs.499, else Rs.49
total	Float	subtotal + deliveryCharge - discount
notes	String?	Optional (payment reference, remarks)
createdAt	DateTime	Auto-set
updatedAt	DateTime	Auto-update

4.9 OrderItem

Field	Type	Constraints
id	Int	PK, auto-increment
orderId	Int	FK → Order
productId	Int	FK → Product
quantity	Int	Ordered quantity
price	Float	Unit price at order time (snapshot)
shade	String?	Variant snapshot
size	String?	Variant snapshot

4.10 Review

Field	Type	Constraints
id	Int	PK, auto-increment
userId	Int	FK → User
productId	Int	FK → Product
rating	Int	1-5 stars
title	String?	Optional headline
body	String?	Optional review text
createdAt	DateTime	Auto-set

Composite unique (userId, productId) — one review per user per product.

4.11 Coupon

Field	Type	Constraints
id	Int	PK, auto-increment
code	String	Unique, uppercase
type	String	PERCENT FLAT

value	Float	Percentage or flat Rs. amount
minOrder	Float	Default: 0, minimum order to apply
maxDiscount	Float?	Optional cap for PERCENT type
usageLimit	Int?	Optional total uses allowed
usedCount	Int	Default: 0
isActive	Boolean	Default: true
expiresAt	DateTime?	Optional expiry
createdAt	DateTime	Auto-set

Seeded Coupons

Code	Type	Value	Min Order	Max Discount
GLAMCART10	PERCENT	10%	Rs.500	Rs.200
FIRST50	FLAT	Rs.50	Rs.299	Rs.50
BEAUTY20	PERCENT	20%	Rs.799	Rs.300
SKINCARE15	PERCENT	15%	Rs.599	Rs.250
FREESHIP	FLAT	Rs.49	Rs.0	Rs.49
MEGA30	PERCENT	30%	Rs.1499	Rs.500

5 Backend API Specification

5.1 Middleware & Config

Middleware	Purpose
CORS	Allowed origins: localhost:3000, localhost:3001
JSON body parser	Parse application/json requests
URL-encoded parser	Parse form data
Request logger	Logs: METHOD /path → STATUS (Xms), colour-coded
JWT auth	Validates Bearer token, attaches req.user
Error handler	Generic 500 JSON response

5.2 Auth Routes — /api/auth

POST /api/auth/register	Auth: None
Body:	
<pre>{ "name": "string", "email": "string", "password": "string", "phone": "optional" }</pre>	
Success Response:	
<pre>201: { "user": { id, email, name, phone, role }, "token": "JWT" }</pre>	
Errors: 400 (validation), 409 (email exists), 500	
POST /api/auth/login	Auth: None
Body:	
<pre>{ "email": "string", "password": "string" }</pre>	
Success Response:	
<pre>200: { "user": { id, email, name, phone, avatar, role, createdAt }, "token": "JWT" }</pre>	
Errors: 400 (validation), 401 (invalid credentials), 500	
GET /api/auth/me	Auth: Required
Success Response:	
<pre>200: { id, email, name, phone, avatar, role, createdAt }</pre>	
Errors: 401, 404, 500	

5.3 Product Routes — /api/products

Query params: category (slug), brand (slug), search (name/desc/tags), minPrice, maxPrice, featured, bestseller, sort (price_asc | price_desc | rating | newest), page (default 1), limit (default 20)

GET /api/products	Auth: None
Success Response:	
<pre>200: { "products": [...], "total": 53, "page": 1, "pages": 3 }</pre>	

GET	/api/products/:slug	Auth: None
Success Response:		
200: Full product with category, brand, last 10 reviews		
Errors: 404, 500		

5.4 Category Routes — /api/categories

Endpoint	Auth	Description
GET /api/categories	None	List top-level categories with product counts and children
GET /api/categories/:slug	None	Get single category with children

5.5 Brand Routes — /api/brands

Endpoint	Auth	Description
GET /api/brands	None	List all brands with product counts
GET /api/brands/:slug	None	Get brand with first 20 products

5.6 Cart Routes — /api/cart

Endpoint	Auth	Description
GET /api/cart	Required	Get user's cart items
POST /api/cart	Required	Add product (increments qty if exists)
PATCH /api/cart/:productId	Required	Update item quantity
DELETE /api/cart/:productId	Required	Remove single item
DELETE /api/cart	Required	Clear entire cart

POST body: { productId, quantity (default 1), shade?, size? } | PATCH body: { quantity } — if < 1, item deleted | Stock check: 400 if qty > product.stock

5.7 Wishlist Routes — /api/wishlist

Endpoint	Auth	Description
GET /api/wishlist	Required	Get user's wishlist
POST /api/wishlist	Required	Add product (upsert, no error if exists)
DELETE /api/wishlist/:productId	Required	Remove product

5.8 Order Routes — /api/orders

Endpoint	Auth	Description
GET /api/orders	Required	Get all user orders (newest first)
GET /api/orders/:id	Required	Get single order (user must own it)

POST /api/orders	Required	Place order from cart
POST /api/orders		Auth: Required
Logic: 1) Cart must not be empty 2) Subtotal = sum(price×qty) 3) Delivery: Rs.0 if subtotal>=Rs.499 else Rs.49 4) Validate coupon → discount 5) Total = subtotal+delivery-discount 6) Create Order+Items+snapshot 7) Clear cart 8) COD→PENDING Razorpay+paymentId→PAID		
Body:		
<pre>{ "addressId": int?, "paymentMethod": "COD UPI CARD NETBANKING", "couponCode": "string?", "razorpayPaymentId": "string?", "notes": "string?" }</pre>		

5.9 User Routes — /api/users

Endpoint	Auth	Description
GET /api/users/profile	Required	Get profile
PATCH /api/users/profile	Required	Update name, phone, avatar
POST /api/users/change-password	Required	Change password
GET /api/users/addresses	Required	List addresses
POST /api/users/addresses	Required	Add address
DELETE /api/users/addresses/:id	Required	Delete address

5.10 Coupon Routes — /api/coupons

Endpoint	Auth	Description
GET /api/coupons	None	List all active coupons (public)
POST /api/coupons/validate	Required	Validate coupon + get discount

Validation steps: 1) Coupon must exist and be active 2) Check expiresAt 3) Check usageLimit 4) Check minOrder 5) PERCENT: min(amount×value%, maxDiscount) | FLAT: value

5.11 Payment Routes — /api/payments

Endpoint	Auth	Description
POST /api/payments/create-order	Required	Create Razorpay order
POST /api/payments/verify	Required	Verify Razorpay signature (HMAC-SHA256)

5.12 Health Check

GET /api/health	Auth: None
Success Response:	
<pre>200: { "status": "ok", "message": "GlamCart API running" }</pre>	

6 Web Frontend Specification

6.1 Pages

URL	Auth	Description
/	No	Home — Hero carousel, featured products, bestsellers
/products	No	Browse with filters, search, pagination
/products/:slug	No	Product info, reviews, add to cart
/login	No	Email/password login
/register	No	Account creation
/cart	Yes	Review cart, apply coupons, proceed to checkout
/checkout	Yes	Address, payment, order review
/orders	Yes	Order history list
/orders/:id	Yes	Full order details
/profile	Yes	Account settings
/wishlist	Yes	Saved products

6.2 Home Page /

- Hero banner with 2-slide auto-carousel (5s interval)
- 6 category quick-links (Makeup, Skincare, Haircare, Fragrance, Bath & Body, Wellness)
- Featured products grid (up to 8) + Bestsellers grid (up to 8)
- Error state with Try Again on API failure + Skeleton loading animation

6.3 Products Page /products

- Filter sidebar (category, price range, featured, bestseller)
- Sort dropdown (Relevance, Price Low-High, Price High-Low, Top Rated, Newest)
- 20 products per page, pagination controls + Search bar with clear button
- Error state shows actual error message + Try Again (not 'No products found' on API failure)
- Skeleton loading (12 placeholder cards)

6.4 Product Detail Page /products/:slug

- Image gallery with clickable thumbnails
- Product info: name, brand link, price, MRP, discount %, stock status
- Quantity selector (1 to stock max) + Add to Bag + Wishlist toggle
- Redirects to /login?redirect=...&addToCart;=:id&qty;=:qty if not logged in
- Description tab, Reviews tab, Related products (same category, up to 4)

6.5 Login Page /login

Query params: redirect (default /), addToCart (product ID), qty (default 1)

After login: signIn() → token in localStorage → optional addToCart API call → fetchCart() → redirect

6.6 Cart Page /cart

- Cart items with quantity controls and remove button
- Delivery: Rs.0 if subtotal $\geq$ Rs.499, else Rs.49
- Coupon section: input (monospace, all-caps) + collapsible coupon list (one-tap apply)
- Green confirmation bar when coupon applied
- Proceeds to /checkout?coupon=CODE

6.7 Checkout Page /checkout

3-step flow:

- Address — Select saved or add new; validates name, phone (10 digits), pincode (6 digits)
- Payment — Select COD, UPI, Card, or Net Banking
- Review — Order items + totals; Place Order button

COD: direct placeOrder() | Razorpay: createRazorpayOrder() → modal → verifyPayment() → placeOrder(paymentId) → PAID

6.8 Header Component

- Sticky top bar with GlamCart logo
- Search bar (submits to /products?search=)
- Cart bag icon with count badge
- User dropdown (Profile / Orders / Wishlist / Logout)
- Category nav links + Responsive hamburger menu on mobile

6.9 ProductCard Component

Displays: image, brand, name, price, MRP (strikethrough), discount badge, rating, stock status, Add to Cart button

7 Mobile App Specification

7.1 App Structure

```
glamcart_flutter/lib/ ■■■ main.dart Entry point, providers setup ■■■ config/ API base URL config
■■■ models/ Dart model classes with fromJson ■■■ providers/ AuthProvider, CartProvider ■■■
services/ ApiService (Dio singleton) ■■■ screens/ One file per screen ■■■ widgets/ Reusable UI
components
```

7.2 Models

Model	Fields / Notes
User	id, name, email, phone?, role — Factory: User.fromJson(json)
Product	id, name, slug, price, mrp?, images, brand?, category?, rating?, stock — Computed: discount%, firstImage
CartItem	id, product (Product), quantity — Computed: total (double)
Order	id, status, paymentStatus, paymentMethod, subtotal, deliveryCharge, discount, total, items[], createdAt
OrderItem	id, productName, productImages, quantity, price
Address	id, name, phone, line1, line2?, city, state, pincode, type, isDefault — Computed: fullAddress

7.3 ApiService

Singleton Dio instance. Config: 10s connect/receive timeout. Request interceptor: attach Bearer token from SharedPreferences. Error interceptor: clear token on 401.

```
ApiService.getErrorMessage(e) connectionTimeout/receiveTimeout → 'Request timed out. Please try
again.' connectionError → 'Server is unavailable. Please try again later.' HTTP error with
data['error'] → that error string otherwise → 'Something went wrong (statusCode)'
```

7.4 Screens

Screen	Description
SplashScreen	Shows GlamCart logo; calls AuthProvider.init() + CartProvider.fetchCart(); auto-navigates to HomeScreen
HomeScreen	Bottom nav (Home/Products/Cart/Orders); IndexedStack; hero banner, category chips, featured/bestseller grids; pull-to-refresh
LoginScreen	Email + password with show/hide toggle; error display; on success: pop + fetch cart
RegisterScreen	Name, email, phone?, password, confirm password; full validation; auto-login on success
ProductsScreen	Shimmer grid; search bar; sort dropdown; category filter; error state with WiFi-off icon + Try Again; pull-to-refresh
ProductDetailScreen	Image carousel with page indicator; Add to Cart requires login; distinguishes 404 vs network errors
CartScreen	Login prompt if unauth; quantity controls; price summary; coupon input; proceed to checkout

CheckoutScreen	Address select + add form; payment method selector; order summary; Razorpay integration; coupon section
OrdersScreen	Login prompt if unauth; orders list with colour-coded badges; thumbnails; error state with WiFi-off; pull-to-refresh

7.5 Providers

Provider	Details
AuthProvider	State: user, loading, isLoggedIn Methods: init() (restore session), login(), register(), logout()
CartProvider	State: items, loading Getters: count, total Methods: fetchCart() (keeps items on failure), addToCart() (throws), updateItem() (throws), removeItem() (throws), clearCart(), reset() (local only)

8 State Management

8.1 Web — React Context

AuthContext:

- `signIn(user, token)` → stores `glamcart_token` + `glamcart_user` in `localStorage` + React state
- `signOut()` → clears `localStorage` + React state
- Auto-init: reads `localStorage` on mount, calls `GET /auth/me` to validate token
- Token cleared automatically on 401 response (Axios interceptor)

CartContext:

- `fetchCart()` — stable `useCallback` (no user in deps); reads token from `localStorage` via `Axios`
- Separate `useEffect` watches user: fetches cart on login, clears on logout
- `addItem`, `updateItem`, `removeItem`, `emptyCart` all call `fetchCart` after mutation
- `cartCount` and `cartTotal` are computed values (not state)

8.2 Mobile — Provider Pattern

- `AuthProvider` and `CartProvider` registered in `main.dart` with `MultiProvider`
- `ChangeNotifier` + `notifyListeners()` pattern
- `context.watch()` for reactive rebuilds
- `context.read()` for one-time reads

9 Validation Rules

Field	Rule	Enforced
Name	Min 2 characters	Frontend + Backend
Email	Regex: ^[\s@]+\.[\s@]+\.[\s@]+\$	Frontend + Backend
Password (register)	Min 8 chars + at least 1 special char	Frontend + Backend
Password (login)	Non-empty, min 6 chars	Frontend
Phone	Exactly 10 digits, numeric	Frontend + Backend
Pincode	Exactly 6 digits, numeric	Frontend + Backend
Quantity	Min 1, max = product.stock	Frontend + Backend

Special characters accepted in password: ! @ # \$ % ^ & * () _ + - = [] { } ; : | , . < > / ? ~ `

10 Security

10.1 Authentication

- JWT tokens with 7-day expiry
- Signed with JWT_SECRET environment variable
- Token sent as Authorization: Bearer <token> header
- All protected routes use authenticate middleware
- 401 response clears token automatically on both web and mobile

10.2 Password Security

- Hashed with bcrypt, salt rounds: 10
- Plaintext password never stored or logged
- Password never returned in any API response

10.3 Data Access

- Users can only access their own cart, orders, wishlist, addresses
- Order detail returns 404 if order belongs to a different user
- Admin role exists in schema but admin-only routes not yet implemented

10.4 API Security

- CORS restricted to localhost:3000 and localhost:3001 in development
- Razorpay payment signature verified with HMAC-SHA256 before order marked PAID
- No sensitive fields (password, internal IDs beyond necessary) returned in responses

10.5 Input Sanitization

- Email validated by regex before DB query
- Prisma ORM used for all DB operations (parameterized queries, no raw SQL injection risk)
- Request body parsed strictly with `express.json()`

11 Key User Flows

11.1 Registration

1. User fills name, email, password, phone (optional)
2. Client validates all fields
3. POST /api/auth/register
4. Backend validates, hashes password, creates User
5. Returns user object + JWT
6. Client stores token, updates auth state
7. Redirects to home

11.2 Login with Pending Cart Add

1. Unauthenticated user clicks 'Add to Bag' on product page
2. Redirected to /login?redirect=/products/:slug&addToCart;=:id&qty;=:qty
3. User logs in → signIn() stores token in localStorage
4. Login page calls raw addToCart(id, qty) API directly (token immediately available)
5. Calls fetchCart() to update bag count in header
6. Redirects to /products/:slug

11.3 Cart → Checkout → Order

1. User adds items to cart
2. Optionally applies coupon (validated in real-time)
3. Proceeds to /checkout?coupon=CODE
4. Checkout page auto-validates coupon from URL param on mount
5. User selects address + payment method
6. COD: placeOrder() → order created, cart cleared, redirect to order detail
7. Razorpay: createRazorpayOrder() → modal → verifyPayment() → placeOrder(razorpayPaymentId) → paymentStatus PAID

11.4 Coupon Application

1. User enters coupon code or selects from list
2. POST /api/coupons/validate with code + cartTotal
3. Backend checks: active, not expired, usage limit, min order met
4. Returns discount amount
5. Cart total updates in real-time
6. Coupon code passed as URL param to checkout

11.5 Session Restore

1. App/page loads
2. Web: AuthContext reads glamcart_token from localStorage → calls GET /auth/me
3. Mobile: AuthProvider reads token from SharedPreferences → calls GET /auth/me
4. If valid: user state restored, cart fetched
5. If 401: token cleared, user logged out

12 Error Handling

12.1 API Error Helper — Web

```
getApiError(err): err.code === 'ECONNABORTED' → 'Request timed out. Please try again.'
!navigator.onLine → 'No internet connection. Please check your network.' no response (server down) →
'Server is unavailable. Please try again later.' HTTP error with data.error → backend error message
other HTTP error → 'Something went wrong (statusCode)'
```

12.2 API Error Helper — Flutter

```
ApiService.getErrorMessage(e): connectionTimeout/receiveTimeout → 'Request timed out. Please try
again.' connectionError → 'Server is unavailable. Please try again later.' HTTP error with
data['error'] → backend error message other → 'Something went wrong (statusCode)'
```

12.3 Error States in UI

Screen	On API Failure
Products page (web)	Error message + Try Again button
Homepage (web)	Error message + Try Again button
Products screen (Flutter)	WiFi-off icon + error message + Try Again
Home screen (Flutter)	WiFi-off icon + error message + Try Again
Orders screen (Flutter)	WiFi-off icon + error message + Try Again
Product detail (Flutter)	Distinguishes 404 vs network error
Add to cart (Flutter)	Red error snackbar on failure

12.4 Request Logger — Backend

Every request logs: METHOD /path → STATUS (Xms) with colour coding: Green = 2xx | Yellow = 4xx | Red = 5xx

13 Known Limitations

Limitation	Detail
Razorpay is test mode only	No real payments processed
No image upload	Product images are URLs only; no file upload supported
No admin panel UI	Admin actions via seed script or API only
Flutter is companion app	Wishlist and profile editing not available in mobile app
No email notifications	Order confirmation emails not implemented
Single currency	INR only, no multi-currency support
No real-time updates	Cart/order status requires manual refresh
CORS is localhost only	Production deployment requires CORS update
No rate limiting	API has no request throttling
Reviews not writable via UI	Reviews exist in DB but no 'Write a Review' form implemented

End of Document • GlamCart SRS v1.0 • 2026-05-08 • Author: Hamja Paldiwalla